

Software Requirements Specification (SRS)

ระบบ **Portfolio** ออนไลน์สำหรับนักศึกษาวิศวกรรมคอมพิวเตอร์

เวอร์ชันเอกสาร	1.0
วันที่จัดทำ	12 กรกฎาคม 2026
เทคโนโลยีที่เสนอ	Next.js, MongoDB
กลุ่มผู้ใช้งานหลัก	นักศึกษาวิศวกรรมคอมพิวเตอร์, ผู้ดูแลระบบ

1. บทนำ

1.1 วัตถุประสงค์ของเอกสาร

เอกสารฉบับนี้จัดทำขึ้นเพื่อกำหนดความต้องการของระบบ Portfolio ออนไลน์สำหรับนักศึกษาวิศวกรรมคอมพิวเตอร์ โดยครอบคลุมความต้องการด้านฟังก์ชันการทำงาน คุณสมบัติของระบบ ขอบเขตการพัฒนา บทบาทของผู้ใช้งาน โครงสร้างข้อมูลเบื้องต้น และข้อกำหนดที่เกี่ยวข้องกับความปลอดภัยและประสิทธิภาพ

เอกสารนี้สามารถใช้เป็นแนวทางสำหรับทีมพัฒนา ทีมออกแบบ UI/UX ผู้ทดสอบระบบ และผู้มีส่วนเกี่ยวข้องในการวางแผนพัฒนาระบบ

1.2 ขอบเขตของระบบ

ระบบ Portfolio ออนไลน์เป็นเว็บแอปพลิเคชันที่เปิดให้นักศึกษาสามารถสร้างและออกแบบหน้า Portfolio ของตนเองได้ผ่านเครื่องมือแก้ไขแบบ Drag and Drop ที่ใช้งานง่าย ไม่ซับซ้อน โดยนักศึกษาสามารถเลือกสไลด์ ปรับข้อความ ปรับขนาดตัวอักษร จัดวางส่วนประกอบพื้นฐาน และเผยแพร่ Portfolio เป็นลิงก์สาธารณะได้

ฝั่งผู้ดูแลระบบสามารถเข้าสู่ระบบเพื่อเพิ่มข้อมูลนักศึกษา จัดการบัญชีผู้ใช้งาน และตรวจสอบ Portfolio ที่นักศึกษาแต่ละคนออกแบบไว้ได้

1.3 เป้าหมายของระบบ

- ให้นักศึกษาสร้าง Portfolio ออนไลน์ได้ด้วยตนเองโดยไม่จำเป็นต้องเขียนโค้ด
- ลดความซับซ้อนของการออกแบบ Portfolio ด้วยเครื่องมือ Drag and Drop พื้นฐาน
- รองรับการเผยแพร่ Portfolio เป็นลิงก์สาธารณะหรือ Portal ส่วนตัว
- ให้ผู้ดูแลระบบสามารถจัดการนักศึกษาและตรวจสอบผลงานได้
- มีระบบ Login เพื่อแยกสิทธิ์ระหว่างนักศึกษาและผู้ดูแลระบบ
- ใช้ Next.js สำหรับพัฒนาเว็บแอปพลิเคชัน และ MongoDB สำหรับจัดเก็บข้อมูล

1.4 คำจำกัดความ

คำศัพท์	ความหมาย
Portfolio	หน้าเว็บแสดงข้อมูล ประวัติ ผลงาน ทักษะ และโปรเจกต์ของนักศึกษา
Drag and Drop	การลากวางองค์ประกอบเพื่อจัดหน้าโดยไม่ต้องเขียนโค้ด

คำศัพท์	ความหมาย
Portal Link	ลิงก์สาธารณะสำหรับเข้าชม Portfolio ของนักศึกษา
Student	นักศึกษาที่สามารถสร้าง แก้ไข และเผยแพร่ Portfolio ของตนเอง
Admin	ผู้ดูแลระบบที่สามารถจัดการบัญชีนักศึกษาและตรวจสอบ Portfolio
Template	รูปแบบหรือธีมเริ่มต้นของ Portfolio
Component/Block	ส่วนประกอบของหน้า Portfolio เช่น ข้อความ รูปภาพ โปรเจกต์ ทักษะ ประวัติการศึกษา

2. ภาพรวมระบบ

2.1 ลักษณะของระบบ

ระบบเป็นเว็บแอปพลิเคชันแบบออนไลน์ สามารถใช้งานผ่านเว็บเบราว์เซอร์ โดยมีส่วนหลักดังนี้

1. ระบบยืนยันตัวตนและจัดการสิทธิ์ผู้ใช้งาน
2. Dashboard สำหรับนักศึกษา
3. Portfolio Editor แบบ Drag and Drop
4. ระบบเลือกสไลด์และปรับแต่งพื้นฐาน
5. ระบบเผยแพร่ Portfolio เป็นลิงก์สาธารณะ
6. Dashboard สำหรับผู้ดูแลระบบ
7. ระบบจัดการข้อมูลนักศึกษา
8. ระบบดูตัวอย่าง Portfolio ของนักศึกษา

2.2 กลุ่มผู้ใช้งาน

2.2.1 นักศึกษา

นักศึกษาสามารถเข้าสู่ระบบ แก้ไขข้อมูลส่วนตัว สร้างและออกแบบหน้า Portfolio เลือกรูปแบบการแสดงผล เพิ่มผลงาน เพิ่มทักษะ ปรับแต่งหน้าตาเบื้องต้น ดูตัวอย่าง และเผยแพร่ Portfolio เป็นลิงก์ได้

2.2.2 ผู้ดูแลระบบ

ผู้ดูแลระบบสามารถเข้าสู่ระบบ เพิ่ม แก้ไข ปิดใช้งาน หรือลบข้อมูลนักศึกษา ดูรายการ Portfolio ของนักศึกษา ตรวจสอบสถานะการเผยแพร่ และเปิดดูตัวอย่าง Portfolio ของนักศึกษาแต่ละคนได้

2.2.3 ผู้เข้าชมทั่วไป

ผู้เข้าชมทั่วไปสามารถเปิดดู Portfolio ผ่านลิงก์สาธารณะที่นักศึกษาเผยแพร่ไว้ โดยไม่จำเป็นต้องเข้าสู่ระบบ

2.3 สภาพแวดล้อมการใช้งาน

- ใช้งานผ่านเว็บเบราว์เซอร์สมัยใหม่ เช่น Google Chrome, Microsoft Edge, Safari, Firefox
- รองรับ Desktop เป็นหลัก และควรรองรับ Tablet/Mobile สำหรับการแสดงผล Portfolio
- ฝั่งพัฒนาระบบใช้ Next.js
- ฐานข้อมูลใช้ MongoDB
- ระบบ Authentication สามารถพัฒนาเองหรือใช้ NextAuth.js/Auth.js ร่วมกับ JWT หรือ Session ได้
- ระบบจัดเก็บรูปภาพสามารถใช้ Cloud Storage ในอนาคต เช่น AWS S3, Cloudinary หรือ Firebase Storage

2.4 ข้อจำกัดเบื้องต้น

- Editor แบบ Drag and Drop ต้องไม่ซับซ้อนเกินไป เพื่อให้นักศึกษาทั่วไปใช้งานได้ง่าย
- ระบบในระยะแรกเน้นการแก้ไข Layout และ Style พื้นฐาน ไม่จำเป็นต้องรองรับการออกแบบอิสระระดับเดียวกับเครื่องมือออกแบบมืออาชีพ
- นักศึกษาแก้ไขได้เฉพาะ Portfolio ของตนเอง
- Admin ไม่ควรแก้ไขเนื้อหา Portfolio แทนนักศึกษาโดยไม่จำเป็น ยกเว้นมี requirement เพิ่มเติมภายหลัง
- Portfolio ที่เผยแพร่แล้วควรมี URL ที่อ่านง่ายและไม่ซ้ำกัน

3. ความต้องการด้านฟังก์ชัน

3.1 ระบบ Authentication และ Authorization

FR-001: เข้าสู่ระบบ

ระบบต้องให้ผู้ใช้งานเข้าสู่ระบบด้วยอีเมล/รหัสนักศึกษาหรือ Username และรหัสผ่าน

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา, Admin

ความสำคัญ: สูง

FR-002: ออกจากระบบ

ระบบต้องให้ผู้ใช้งานออกจากระบบได้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา, Admin

ความสำคัญ: สูง

FR-003: แยกสิทธิ์ตามบทบาทผู้ใช้งาน

ระบบต้องแยกสิทธิ์การใช้งานระหว่าง Student และ Admin โดย Student เข้าถึงได้เฉพาะข้อมูลและ Portfolio ของตนเอง ส่วน Admin เข้าถึงหน้าจัดการนักศึกษาและหน้าตรวจสอบ Portfolio ได้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา, Admin

ความสำคัญ: สูง

FR-004: เปลี่ยนรหัสผ่าน

ระบบควรให้นักศึกษาและ Admin สามารถเปลี่ยนรหัสผ่านของตนเองได้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา, Admin

ความสำคัญ: กลาง

3.2 ระบบจัดการนักศึกษาโดย Admin

FR-005: เพิ่มนักศึกษา

Admin ต้องสามารถเพิ่มข้อมูลนักศึกษาเข้าสู่ระบบได้ โดยข้อมูลขั้นต่ำประกอบด้วย รหัสนักศึกษา ชื่อ นามสกุล อีเมล สาขา ชั้นปี และสถานะบัญชี

ผู้ใช้งานที่เกี่ยวข้อง: Admin

ความสำคัญ: สูง

FR-006: ดูรายการนักศึกษา

Admin ต้องสามารถดูรายชื่อนักศึกษาทั้งหมดในระบบได้ พร้อมข้อมูลสำคัญ เช่น รหัสนักศึกษา ชื่อ อีเมล สถานะบัญชี และสถานะ Portfolio

ผู้ใช้งานที่เกี่ยวข้อง: Admin

ความสำคัญ: สูง

FR-007: ค้นหาและกรองนักศึกษา

Admin ควรสามารถค้นหานักศึกษาจากชื่อ รหัสนักศึกษา หรืออีเมล และกรองตามชั้นปี สาขา หรือสถานะ Portfolio ได้

ผู้ใช้งานที่เกี่ยวข้อง: Admin

ความสำคัญ: กลาง

FR-008: แก้ไขข้อมูลนักศึกษา

Admin ต้องสามารถแก้ไขข้อมูลพื้นฐานของนักศึกษาได้

ผู้ใช้งานที่เกี่ยวข้อง: Admin

ความสำคัญ: สูง

FR-009: ปิดใช้งานบัญชีนักศึกษา

Admin ต้องสามารถปิดใช้งานบัญชีนักศึกษาได้ โดยบัญชีที่ถูกปิดใช้งานจะไม่สามารถเข้าสู่ระบบได้

ผู้ใช้งานที่เกี่ยวข้อง: Admin

ความสำคัญ: กลาง

FR-010: รีเซตรหัสผ่านนักศึกษา

Admin ควรสามารถตั้งรหัสผ่านใหม่หรือส่งลิงก์รีเซตรหัสผ่านให้นักศึกษาได้

ผู้ใช้งานที่เกี่ยวข้อง: Admin

ความสำคัญ: กลาง

3.3 Dashboard สำหรับนักศึกษา

FR-011: ดูภาพรวม Portfolio

นักศึกษาต้องสามารถดูสถานะ Portfolio ของตนเองได้ เช่น ยังไม่สร้าง ฉบับร่าง เผยแพร่แล้ว หรือปิดเผยแพร่

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-012: แก้ไขข้อมูลส่วนตัว

นักศึกษาต้องสามารถแก้ไขข้อมูลส่วนตัวที่ใช้แสดงใน Portfolio ได้ เช่น ชื่อ รูปโปรไฟล์ คำแนะนำตัว ช่องทางติดต่อ GitHub LinkedIn และเว็บไซต์ส่วนตัว

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-013: เข้าสู่หน้า Portfolio Editor

นักศึกษาต้องสามารถเปิดหน้า Editor เพื่อออกแบบและแก้ไข Portfolio ได้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-014: ดูตัวอย่าง Portfolio

นักศึกษาต้องสามารถดูตัวอย่าง Portfolio ก่อนเผยแพร่ได้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

3.4 Portfolio Editor แบบ Drag and Drop

FR-015: เพิ่ม Section หรือ Block

ระบบต้องให้นักศึกษาเพิ่มส่วนประกอบของ Portfolio ได้ เช่น

- Header/Profile
- About Me
- Education
- Skills
- Projects
- Experience
- Certificates
- Contact
- Custom Text
- Image

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-016: ลากเพื่อจัดลำดับ Section

ระบบต้องให้นักศึกษาลากและวาง Section เพื่อเปลี่ยนลำดับการแสดงผลได้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-017: แก้ไขเนื้อหาภายใน Section

ระบบต้องให้นักศึกษาแก้ไขข้อความ รูปภาพ ลิงก์ รายละเอียดโปรเจกต์ รายการทักษะ และข้อมูลอื่น ๆ ภายใน Section ได้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-018: ลบ Section

ระบบต้องให้นักศึกษาลบ Section ที่ไม่ต้องการได้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-019: ซ่อนหรือแสดง Section

ระบบควรให้นักศึกษาสามารถซ่อน Section ซ้ำคร่าวได้โดยไม่ต้องลบทิ้ง

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: กลาง

FR-020: ปรับ Style พื้นฐาน

ระบบต้องให้นักศึกษาปรับแต่ง Style พื้นฐานได้ เช่น

- เลือกธีมหรือ Template
- เลือกสีหลักของ Portfolio
- ปรับขนาดตัวอักษรพื้นฐาน
- เลือกรูปแบบตัวอักษรจากชุดที่ระบบกำหนด
- ปรับรูปแบบการจัดวางเบื้องต้น เช่น แบบเรียงง่าย แบบเน้นโปรเจกต์ หรือแบบเน้นโปรไฟล์

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-021: บันทึกฉบับร่าง

ระบบต้องสามารถบันทึกการออกแบบ Portfolio เป็น Draft ได้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-022: บันทึกอัตโนมัติ

ระบบควรมีการบันทึกอัตโนมัติเป็นระยะ เพื่อลดความเสี่ยงข้อมูลสูญหาย

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: กลาง

FR-023: แสดง Preview แบบ Real-time หรือ Near Real-time

ระบบควรแสดงตัวอย่าง Portfolio ขณะนักศึกษาแก้ไขได้ โดยอาจเป็น Preview ด้านข้างหรือปุ่ม Preview แยกหน้า

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: กลาง

3.5 ระบบเผยแพร่ Portfolio

FR-024: เผยแพร่ Portfolio

นักศึกษาต้องสามารถกดเผยแพร่ Portfolio ของตนเองได้ เมื่อระบบเผยแพร่สำเร็จจะสร้างลิงก์สาธารณะให้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-025: สร้าง URL เฉพาะของนักศึกษา

ระบบต้องสร้าง URL ที่ไม่ซ้ำกันสำหรับ Portfolio แต่ละคน เช่น `/portfolio/student-id` หรือ `/p/custom-slug`

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา, ผู้เข้าชมทั่วไป

ความสำคัญ: สูง

FR-026: แก้ไข Slug

ระบบควรให้นักศึกษากำหนด Slug ของ Portfolio ได้ภายใต้เงื่อนไขที่ไม่ซ้ำกับผู้อื่น และผ่านการตรวจสอบรูปแบบที่ถูกต้อง

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: กลาง

FR-027: ปิดเผยแพร่ Portfolio

นักศึกษาต้องสามารถปิดการเผยแพร่ Portfolio ได้ โดยลิงก์สาธารณะจะไม่สามารถเปิดดูเนื้อหาได้

ผู้ใช้งานที่เกี่ยวข้อง: นักศึกษา

ความสำคัญ: สูง

FR-028: ผู้เข้าชมเปิดดู Portfolio

ผู้เข้าชมทั่วไปต้องสามารถเปิดดู Portfolio ผ่านลิงก์สาธารณะได้โดยไม่ต้อง Login

ผู้ใช้งานที่เกี่ยวข้อง: ผู้เข้าชมทั่วไป

ความสำคัญ: สูง

3.6 ระบบตรวจสอบ Portfolio โดย Admin

FR-029: ดูรายการ Portfolio

Admin ต้องสามารถดูรายการ Portfolio ของนักศึกษาทั้งหมดได้ พร้อมสถานะ เช่น ยังไม่สร้าง ฉบับร่าง เผยแพร่แล้ว หรือปิดเผยแพร่

ผู้ใช้งานที่เกี่ยวข้อง: Admin

ความสำคัญ: สูง

FR-030: ดู Portfolio ของนักศึกษา

Admin ต้องสามารถเปิดดูตัวอย่าง Portfolio ของนักศึกษาแต่ละคนได้

ผู้ใช้งานที่เกี่ยวข้อง: Admin

ความสำคัญ: สูง

FR-031: กรอง Portfolio ตามสถานะ

Admin ควรสามารถกรอง Portfolio ตามสถานะการเผยแพร่ ชั้นปี หรือสาขาได้

ผู้ใช้งานที่เกี่ยวข้อง: Admin

ความสำคัญ: กลาง

FR-032: ดูวันที่แก้ไขล่าสุด

ระบบควรแสดงวันที่นักศึกษาแก้ไข Portfolio ล่าสุด เพื่อให้ Admin ตรวจสอบความเคลื่อนไหวได้

ผู้ใช้งานที่เกี่ยวข้อง: Admin

ความสำคัญ: กลาง

4. ความต้องการด้านข้อมูล

4.1 ข้อมูลผู้ใช้งาน

ระบบต้องจัดเก็บข้อมูลผู้ใช้งานอย่างน้อยดังนี้

Field	รายละเอียด
_id	รหัสภายในของผู้ใช้งาน
studentId	รหัสนักศึกษา
firstName	ชื่อ
lastName	นามสกุล
email	อีเมล
passwordHash	รหัสผ่านที่ผ่านการ Hash
role	บทบาท เช่น student, admin
department	สาขาหรือภาควิชา

Field	รายละเอียด
year	ชั้นปี
status	สถานะบัญชี เช่น active, inactive
createdAt	วันที่สร้างบัญชี
updatedAt	วันที่แก้ไขล่าสุด

4.2 ข้อมูล Portfolio

ระบบต้องจัดเก็บข้อมูล Portfolio อย่างน้อยดังนี้

Field	รายละเอียด
_id	รหัส Portfolio
userId	รหัสนักศึกษาที่เป็นเจ้าของ
title	ชื่อ Portfolio
slug	URL Slug
status	สถานะ เช่น draft, published, unpublished
theme	ธีมหรือ Template ที่เลือก
styleSettings	ค่าการปรับแต่ง เช่น สีหลัก ฟอนต์ ขนาดตัวอักษร
sections	รายการ Section/Block ทั้งหมด
publishedAt	วันที่เผยแพร่
createdAt	วันที่สร้าง
updatedAt	วันที่แก้ไขล่าสุด

4.3 โครงสร้าง Section ตัวอย่าง

```
{
  "id": "section-001",
  "type": "projects",
  "title": "Projects",
  "visible": true,
  "order": 1,
  "content": {
    "items": [
      {
        "name": "Smart Attendance System",
        "description": "ระบบเช็คชื่อด้วย QR Code",
        "techStack": ["Next.js", "MongoDB", "Node.js"],
        "link": "https://example.com"
      }
    ]
  }
}
```

```
    ]  
  }  
}
```

5. ความต้องการด้าน Non-Functional

5.1 ประสิทธิภาพ

- หน้า Dashboard ควรโหลดภายใน 3 วินาทีภายใต้การใช้งานปกติ
- หน้า Portfolio สาธารณะควรโหลดได้รวดเร็ว
- การบันทึก Portfolio ควรตอบสนองภายใน 1-2 วินาทีหลังผู้ใช้กดบันทึก
- ระบบควรรองรับนักศึกษาอย่างน้อย 500-1,000 บัญชีในระยะเริ่มต้น

5.2 ความปลอดภัย

- รหัสผ่านต้องจัดเก็บแบบ Hash เช่น bcrypt หรือ argon2
- ต้องมีการตรวจสอบสิทธิ์ก่อนเข้าถึงหน้า Dashboard, Editor และ Admin
- ต้องป้องกันไม่ให้นักศึกษาเข้าถึงหรือแก้ไข Portfolio ของผู้อื่น
- ต้องตรวจสอบข้อมูลนำเข้าจากผู้ใช้เพื่อป้องกัน XSS และ Injection
- ควรกำหนด Session Expiration หรือ Token Expiration
- Admin Route ต้องถูกป้องกันด้วย Role-based Access Control

5.3 ความพร้อมใช้งาน

- ระบบควรสามารถใช้งานได้ต่อเนื่องผ่านเว็บเบราว์เซอร์
- หากเกิดข้อผิดพลาดระหว่างบันทึก ควรแสดงข้อความแจ้งเตือนที่เข้าใจง่าย
- ควรมี Loading State ระหว่างดึงข้อมูลหรือบันทึกข้อมูล

5.4 การใช้งานง่าย

- Editor ต้องใช้งานง่าย เหมาะกับนักศึกษาที่ไม่มีพื้นฐานการออกแบบเว็บ
- การลากวางต้องมี Feedback ทางสายตา เช่น แสดงตำแหน่งที่จะวาง
- ปุ่มสำคัญ เช่น Save, Preview, Publish ต้องเห็นชัดเจน
- ข้อความในระบบควรใช้ภาษาที่เข้าใจง่าย

5.5 การรองรับอุปกรณ์

- Dashboard และ Editor ควรรองรับ Desktop เป็นหลัก
- หน้า Portfolio สาธารณะต้องแสดงผลได้ดีบน Desktop, Tablet และ Mobile
- Layout ต้องเป็น Responsive Design

5.6 การบำรุงรักษา

- โค้ดควรแยกโครงสร้างชัดเจน เช่น app, components, lib, models, services
- ควรมี TypeScript เพื่อช่วยลดข้อผิดพลาดด้านชนิดข้อมูล
- ควรมี Schema Validation เช่น Zod สำหรับตรวจสอบข้อมูล
- ควรออกแบบ MongoDB Schema ให้ขยายต่อได้ในอนาคต

6. ข้อกำหนดด้าน UI/UX

6.1 หน้าจอ Login

หน้าจอ Login ต้องประกอบด้วย

- ช่องกรอก Username/Email/Student ID
- ช่องกรอกรหัสผ่าน
- ปุ่มเข้าสู่ระบบ
- ข้อความแจ้งเตือนเมื่อเข้าสู่ระบบไม่สำเร็จ

6.2 Student Dashboard

Student Dashboard ควรประกอบด้วย

- สถานะ Portfolio
- ปุ่มแก้ไข Portfolio
- ปุ่มดูตัวอย่าง
- ปุ่มเผยแพร่หรือปิดเผยแพร่
- ข้อมูลโปรไฟล์โดยย่อ
- วันที่แก้ไขล่าสุด

6.3 Portfolio Editor

Portfolio Editor ควรประกอบด้วย

- Sidebar สำหรับเลือก Section/Block
- พื้นที่ Canvas สำหรับจัดวาง Portfolio
- Panel สำหรับแก้ไขรายละเอียดและ Style
- ปุ่ม Save
- ปุ่ม Preview
- ปุ่ม Publish

6.4 Admin Dashboard

Admin Dashboard ควรประกอบด้วย

- จำนวนผู้ใช้งานทั้งหมด
- จำนวน Portfolio ที่เผยแพร่แล้ว
- จำนวน Portfolio ที่ยังเป็น Draft
- ตารางรายชื่อนักศึกษา
- ตัวค้นหาและตัวกรอง
- ปุ่มเพิ่มนักศึกษา

7. Use Case หลัก

7.1 นักศึกษารสร้าง Portfolio

Actor: นักศึกษา

Precondition: นักศึกษามีบัญชีและเข้าสู่ระบบแล้ว

Main Flow:

1. นักศึกษาเข้าสู่ Dashboard
2. นักศึกษาคลิกปุ่มแก้ไข Portfolio
3. ระบบเปิดหน้า Portfolio Editor
4. นักศึกษาเลือก Template หรือ Theme
5. นักศึกษาเพิ่ม Section ที่ต้องการ
6. นักศึกษาแก้ไขเนื้อหาและจัดลำดับด้วย Drag and Drop
7. นักศึกษาคลิก Save
8. ระบบบันทึกเป็น Draft
9. นักศึกษาคลิก Preview เพื่อตรวจสอบ
10. นักศึกษาคลิก Publish
11. ระบบสร้างลิงก์สาธารณะให้

Postcondition: Portfolio ถูกเผยแพร่และสามารถเปิดดูผ่านลิงก์ได้

7.2 Admin เพิ่มนักศึกษา

Actor: Admin

Precondition: Admin เข้าสู่ระบบแล้ว

Main Flow:

1. Admin เข้าหน้า Admin Dashboard
2. Admin คลิกปุ่มเพิ่มนักศึกษา
3. ระบบแสดงฟอร์มเพิ่มนักศึกษา
4. Admin กรอกข้อมูลนักศึกษา
5. Admin กดบันทึก
6. ระบบตรวจสอบข้อมูล
7. ระบบสร้างบัญชีนักศึกษา
8. ระบบแสดงนักศึกษาในรายการ

Postcondition: นักศึกษาสามารถเข้าสู่ระบบได้ตามข้อมูลบัญชีที่กำหนด

7.3 Admin ตรวจสอบ Portfolio

Actor: Admin

Precondition: Admin เข้าสู่ระบบแล้ว

Main Flow:

1. Admin เข้าหน้า Portfolio Management
2. ระบบแสดงรายการ Portfolio ทั้งหมด
3. Admin ค้นหาหรือกรองนักศึกษา
4. Admin เลือก Portfolio ที่ต้องการดู
5. ระบบแสดงตัวอย่าง Portfolio ของนักศึกษา

Postcondition: Admin สามารถตรวจสอบรูปแบบและสถานะ Portfolio ได้

8. ข้อกำหนดด้าน API เบื้องต้น

8.1 Authentication API

Method	Endpoint	รายละเอียด
POST	/api/auth/login	เข้าสู่ระบบ
POST	/api/auth/logout	ออกจากระบบ
POST	/api/auth/change-password	เปลี่ยนรหัสผ่าน

8.2 Student API

Method	Endpoint	รายละเอียด
GET	/api/students	Admin ดึงรายชื่อนักศึกษา
POST	/api/students	Admin เพิ่มนักศึกษา
GET	/api/students/:id	Admin ดูข้อมูลนักศึกษารายคน
PUT	/api/students/:id	Admin แก้ไขข้อมูลนักศึกษา
PATCH	/api/students/:id/status	Admin เปลี่ยนสถานะบัญชี

8.3 Portfolio API

Method	Endpoint	รายละเอียด
GET	/api/portfolio/me	นักศึกษาดึง Portfolio ของตนเอง
PUT	/api/portfolio/me	นักศึกษابันทึกรายการ Portfolio ของตนเอง
POST	/api/portfolio/me/ publish	นักศึกษาเผยแพร่ Portfolio
POST	/api/portfolio/me/ unpublish	นักศึกษายกเลิกเผยแพร่ Portfolio
GET	/api/portfolios	Admin ดึงรายการ Portfolio ทั้งหมด
GET	/api/portfolios/:id	Admin ดู Portfolio รายการ
GET	/p/:slug	ผู้เข้าชมเปิด Portfolio สาธารณะ

9. ข้อกำหนดด้าน Validation

9.1 ข้อมูลนักศึกษา

- รหัสนักศึกษาต้องไม่ว่างและต้องไม่ซ้ำ
- อีเมลต้องอยู่ในรูปแบบที่ถูกต้องและต้องไม่ซ้ำ
- ชื่อและนามสกุลต้องไม่ว่าง
- Role ต้องเป็นค่าที่ระบบกำหนดเท่านั้น

9.2 ข้อมูล Portfolio

- Slug ต้องไม่ซ้ำและใช้ได้เฉพาะตัวอักษรภาษาอังกฤษ ตัวเลข และเครื่องหมายขีดกลาง
- Section ต้องมี id, type, order, และ content
- Section type ต้องเป็นค่าที่ระบบรองรับ
- Link ที่กรอกใน Portfolio ต้องอยู่ในรูปแบบ URL ที่ถูกต้อง
- ข้อความที่ผู้ใช้กรอกต้องถูก sanitize ก่อนแสดงผลในหน้าสาธารณะ

10. ข้อกำหนดด้านสิทธิ์การเข้าถึง

Feature	Student	Admin	Public Visitor
Login	ได้	ได้	ไม่ได้
แก้ไข Portfolio ตนเอง	ได้	ไม่ได้โดยค่าเริ่มต้น	ไม่ได้
ดู Portfolio ตนเอง	ได้	ได้	ได้ เฉพาะที่เผยแพร่

Feature	Student	Admin	Public Visitor
เพิ่มนักศึกษา	ไม่ได้	ได้	ไม่ได้
แก้ไขข้อมูลนักศึกษา	ไม่ได้	ได้	ไม่ได้
ดูรายชื่อ Portfolio ทั้งหมด	ไม่ได้	ได้	ไม่ได้
เปิดลิงก์ Portfolio สาธารณะ	ได้	ได้	ได้

11. Acceptance Criteria

11.1 ระบบ Login

- ผู้ใช้งานที่มีบัญชีถูกต้องสามารถเข้าสู่ระบบได้
- ผู้ใช้งานที่กรอกรหัสผิดไม่สามารถเข้าสู่ระบบได้
- ผู้ใช้งานที่ถูกปิดบัญชีไม่สามารถเข้าสู่ระบบได้
- Student และ Admin ถูกพาไปยัง Dashboard ที่เหมาะสมกับบทบาทของตนเอง

11.2 ระบบ Admin

- Admin สามารถเพิ่มนักศึกษาใหม่ได้
- ระบบป้องกันการเพิ่มรหัสนักศึกษาหรืออีเมลซ้ำ
- Admin สามารถดูรายชื่อนักศึกษาและค้นหานักศึกษาได้
- Admin สามารถเปิดดู Portfolio ของนักศึกษาแต่ละคนได้

11.3 Portfolio Editor

- นักศึกษาสามารถเพิ่ม Section ได้
- นักศึกษาสามารถลากเพื่อเปลี่ยนลำดับ Section ได้
- นักศึกษาสามารถแก้ไขเนื้อหาใน Section ได้
- นักศึกษาสามารถเลือก Theme และปรับ Style พื้นฐานได้
- นักศึกษาสามารถบันทึก Portfolio เป็น Draft ได้

11.4 การเผยแพร่ Portfolio

- นักศึกษาสามารถเผยแพร่ Portfolio ได้
- ระบบสร้างลิงก์ Portfolio ที่ไม่ซ้ำกัน
- ผู้เข้าชมสามารถเปิดดู Portfolio ที่เผยแพร่แล้วได้
- Portfolio ที่ถูกปิดเผยแพร่ต้องไม่แสดงเนื้อหาแก่ผู้เข้าชมทั่วไป

12. ข้อเสนอแนะด้านเทคนิค

12.1 Frontend

- ใช้ Next.js App Router
- ใช้ TypeScript
- ใช้ React Component แยกตาม Section ของ Portfolio
- ใช้ Drag and Drop Library เช่น `@dnd-kit/core` หรือ `react-beautiful-dnd`
- ใช้ Tailwind CSS หรือ CSS Module สำหรับจัดการ Style
- ใช้ Form Validation เช่น React Hook Form ร่วมกับ Zod

12.2 Backend

- ใช้ Next.js API Routes หรือ Route Handlers
- ใช้ MongoDB ร่วมกับ Mongoose หรือ MongoDB Native Driver
- ใช้ bcrypt/argon2 สำหรับ Hash Password
- ใช้ Session หรือ JWT สำหรับ Authentication
- ใช้ Middleware ตรวจสอบสิทธิ์ Student/Admin

12.3 Database

- Collection หลัก ได้แก่ `users`, `portfolios`, `sessions` หรือ `accounts`
- ควรทำ Index ให้กับ `email`, `studentId`, และ `slug`
- ควรจัดเก็บ Portfolio เป็น JSON document เพื่อรองรับการแก้ไข Section แบบยืดหยุ่น

13. สรุป

ระบบ Portfolio ออนไลน์สำหรับนักศึกษาวิศวกรรมคอมพิวเตอร์มีเป้าหมายเพื่อช่วยให้นักศึกษาสร้างหน้าแสดงผลงานของตนเองได้ง่าย สวยงาม และเผยแพร่เป็นลิงก์สาธารณะได้ โดยมีเครื่องมือแก้ไขแบบ Drag and Drop ที่ไม่ซับซ้อน รองรับการปรับแต่ง Style พื้นฐาน และมีระบบ Admin สำหรับจัดการนักศึกษาและตรวจสอบ Portfolio

ระบบนี้เหมาะสำหรับการพัฒนาด้วย Next.js และ MongoDB เนื่องจากสามารถสร้างเว็บแอปพลิเคชันแบบ Full-stack ได้ในโปรเจกต์เดียว รองรับการจัดเก็บข้อมูลแบบ JSON ที่ยืดหยุ่น และเหมาะกับโครงสร้างข้อมูล Portfolio ที่มี Section หลากหลายรูปแบบ